

Lab 1 - Fast I/O

Slow vs. Fast I/O

Slow I/O	Slow I/O
<pre> t = int(input()) for i in range(t): n = int(input()) if n % 2 == 0: print(f"{n} is an Even number.") else: print(f"{n} is an Odd number.") </pre>	<pre> import java.util.*; public class Main { public static void main(String[] args) { Scanner sc = new Scanner(System.in); int t = sc.nextInt(); // number of test cases for (int i = 0; i < t; i++) { int n = sc.nextInt(); // read the number if (n % 2 == 0) System.out.println(n + " is an Even number."); else System.out.println(n + " is an Odd number."); } sc.close(); } } </pre>
Fast I/O	Fast I/O
<pre> import sys data = sys.stdin.read().strip().split() t = int(data[0]) nums = map(int, data[1:]) out = [] for n in nums: if n % 2 == 0: out.append(f"{n} is an Even number.") else: out.append(f"{n} is an Odd number.") sys.stdout.write("\n".join(out)) </pre>	<pre> import java.io.*; import java.util.*; public class Main { public static void main(String[] args) throws IOException { BufferedReader br = new BufferedReader(new InputStreamReader(System.in)); StringBuilder sb = new StringBuilder(); int t = Integer.parseInt(br.readLine().trim()); for (int i = 0; i < t; i++) { int n = Integer.parseInt(br.readLine().trim()); if (n % 2 == 0) sb.append(n).append(" is an Even number.").append("\n"); else sb.append(n).append(" is an Odd number.").append("\n"); } System.out.print(sb.toString()); } } </pre>

Fast I/O in details

import sys	import java.io.*; import java.util.*;
------------	--

```

# Example: reading a single integer
# sys.stdin.readline() reads one line from standard input (the console or judge).
# The result is a string with a newline at the end, so you usually convert it.
# int() converts that string to an integer.
# After this line, n holds the integer value entered by the user

n = int(sys.stdin.readline())

# Example: reading multiple integers in a line
# sys.stdin.readline() reads the next line as a string.
# .split() splits the string into a list of substrings separated by whitespace.
# map(int, ...) converts each substring into an integer.
# a, b, c = assigns the first three integers to variables a, b, and c.

a, b, c = map(int, sys.stdin.readline().split())

# sys.stdout.write() writes directly to standard output.
# It only accepts strings, so str(n) converts the integer to a string.
# '\n' adds a newline, because sys.stdout.write does not automatically add one
like print() does.

sys.stdout.write(str(n) + '\n')

# For multiple outputs, it's faster to build a list of strings and print them at once
# output = [] creates an empty list to store strings.
# for i in range(1, 6): loops through numbers 1 to 5.
# output.append(str(i)) converts each number to a string and adds it to the list
# "\n".join(output) joins all strings in the list with a newline between each.
# sys.stdout.write(...) + "\n" prints all lines at once with a final newline.

output = []
for i in range(1, 6):
    output.append(str(i))
sys.stdout.write("\n".join(output) + "\n")

```

```

public class Main {
    public static void main(String[] args) {

        // BufferedReader is used for fast input from standard input
        // StringTokenizer is a utility to split a string into tokens, usually separated by
        whitespace.

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        // Example: reading a single integer
        // br.readLine() reads a full line as a String
        // Integer.parseInt() converts that string to an integer
        // After this line, n holds the integer value entered by the user

        int n = Integer.parseInt(br.readLine());

        // Example: reading multiple integers in a line
        // br.readLine() reads the next line as a string
        // StringTokenizer splits the line into tokens separated by whitespace
        // Integer.parseInt() converts each token to an integer
        // a, b, c are assigned the first three integers from the line

        st = new StringTokenizer(br.readLine());
        int a = Integer.parseInt(st.nextToken());
        int b = Integer.parseInt(st.nextToken());
        int c = Integer.parseInt(st.nextToken());

        // BufferedWriter writes to standard output efficiently
        // bw.write() writes a string (integers are converted to strings automatically)
        // "\n" is added because BufferedWriter does not automatically add a newline

        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
        bw.write(n + "\n");

        // For multiple outputs, it's faster to use StringBuilder and print all at once
        // StringBuilder sb stores multiple output lines
        // Loop through numbers 1 to 5 and append each number followed by a newline

        StringBuilder sb = new StringBuilder();
        for (int i = 1; i <= 5; i++) {
            sb.append(i).append("\n");
        }
        bw.write(sb.toString());

        // Flush the buffer to ensure all output is printed

        bw.flush();
    }
}

```